Algorithms

SHORTEST PATHS

Last Week

- DFS on directed graphs
- BFS

This Week

- Shortest paths (SP):
  - Relax
  - SP in DAGs
  - Dijkstra's algorithm

Single Source Shortest Path Algorithms

Algorithm	Input	Time Complexity
BFS	Unweighted graph	$O(n+m)$
DAG -- SP	Weighted directed acyclic graph Arbitrary weights	$O(n+m)$
Dijkstra	Weighted graph Non-negative weights	$O((m+n)\log n)$ – Min Heap $O(n^2)$- array of v.d $O(m+n\log n)$ – Fibonacci Heap
Bellman-Ford	Weighted graph Arbitrary weights	$O(n \cdot m)$

Shortest Path Algorithms

The input to all the algorithms is a directed graph G and a source vertex $s \in V$. The output is the shortest path from s to each $v \in V$.

The vertices in G have the following attributes:

- $v.d$ – holds the length of shortest path from s to v detected so far
- $v.\pi$ – holds the parent of v in the current shortest path from s to v

Initialization & Relaxation

Algorithm 1 INIT

```
1: for each vertex v ∈ G.V do
2:     v.d = ∞
3:     v.π = NIL
4: s.d = 0
```

Complexity? O(|V|)

Algorithm 2 RELAX(u,v)

```
1: if v.d > u.d + w(u, v) then
2:     v.d = u.d + w(u, v)
3:     v.π = u
```

Note: (u,v) must be in E

Exercise 1

Claim: given a weighted, directed graph G, let $p = v_0, v_1, \dots, v_k$ be a shortest path from v_0 to v_k. For any i, j such that $0 \leq i \leq j \leq k$, let $p_{i,j} = v_i, \dots, v_j$ be a sub-path of P. Then, $p_{i,j}$ is a shortest path from v_i to v_j.

Exercise 1 - Solution

Claim: given a weighted, directed graph G, let $p = v_0, v_1, \dots, v_k$ be a shortest path from v_0 to v_k. For any i, j such that $0 \leq i \leq j \leq k$, let $p_{i,j} = v_i, \dots, v_j$ be a sub-path of P. Then, $p_{i,j}$ is a shortest path from v_i to v_j.

Proof:

Decompose $p = v_0, \dots, v_i, \dots v_j, \dots, v_k$ into $p = p_{0,i} + p_{i,j} + p_{j,k}$.

Note that $w(p) = w(p_{0,i}) + w(p_{i,j}) + w(p_{j,k})$.

Assume there is a path $p'_{i,j}$ such that $w(p'_{i,j}) < w(p_{i,j})$.

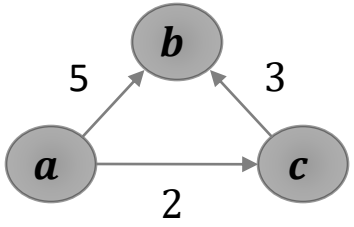
Then, the path $p_{0,i} - p'_{i,j} - p_{j,k}$ is a path from v_0 to v_k whose weight is less than $w(p)$, contradicting the assumption that p is the shortest path from v_0 to v_k.

Exercise 2

For each of the following, prove or give a counter-example:

1. The shortest path in a graph is unique.

False.



Exercise 2

2. The shortest path in an undirected tree is unique.

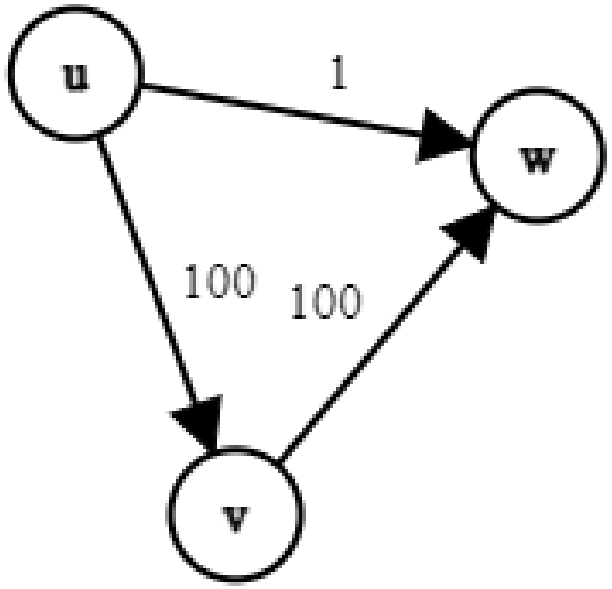
True.

A tree is a graph with a unique path between every pair of vertices thus the shortest path is also unique.

Exercise 2

3. Let p_1 be a shortest path in G from u to v and let p_2 be a shortest path in G from v to w. Then, $p_1 p_2$ is a shortest path from u to w.

False.



DAG Shortest Path Algorithm

The main idea behind this algorithm is utilizing the topological sort to make sure that for every path

$$p = e_1 e_2 \dots e_k$$

in the graph, the edges of p are relaxed according to their order in p.

This is property $*$ for p.

Algorithm 3 DAG-SP(G, w, s)

```
1: INIT(G, s)
2: Q = TOPOSORT(G)
3: while Q ≠ ∅ do
4:     u = DEQUEUE(Q)
5:     for each vertex v ∈ u.adj do
6:         RELAX(u, v, w)
```

Complexity? O(|V|+|E|)

Dijkstra's Algorithm

The main idea behind Dijkstra is to relax each edge once, such that property $*$ is valid for every shortest path p.

Since all edges have positive weights, relaxing the incident edges of the closest vertex will guarantee that once a vertex v is relaxed and removed from the queue, the final distance of v will never change.

Complexity?

Depends on the implementation

O(|E| ·Relax + |V|·EXTRACT-MIN) = O(|E| + |V|log|V|)

Algorithm 4 DIJKSTRA(G, w, s)

```
1: INIT(G, s)
2: S = ∅
3: Q = V
4: while Q ≠ ∅ do
5:     u = EXTRACT-MIN(Q)
6:     S = S ∪ {u}
7:     for each vertex v ∈ u.adj do
8:         RELAX(u, v, w)
```

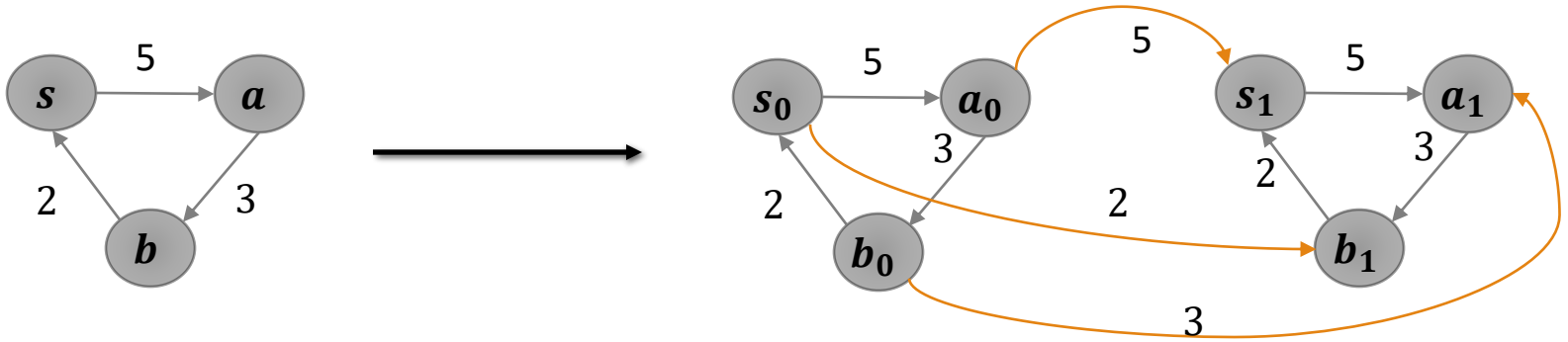
Exercise 3

Let $G = (V, E)$ be a directed graph with positive weights. A path is said to be "almost-valid" if it traverses exactly one edge in its opposite direction. Suggest an algorithm that finds the shortest path from s to all $v \in V$ when we allow **both** valid and almost-valid paths.

Exercise 3 - Solution

Intuition:

Build two copies of the graph, such that traversing an edge in its opposite direction will move us from the first copy to the second copy



Exercise 3 - Solution

Algorithm:

1. Build the following graph: G' = (V', E') such that:
   - For all $v \in V$ we make two copies in G': $v^0, v^1 \in V'$
   - For all $(u, v) \in E$ we make three copies in G':
     - $(u^0, v^0) \in E'$
     - $(u^1, v^1) \in E'$
     - $(v^0, u^1) \in E'$

   w (u, v) = w (u^0, v^0)= w$(u^1, v^1) =$ w(v^0, u^1)
2. Perform Dijkstra on G' from s^0 as a source.
3. For every $v \in V$ return $\min[d(s^0, v^0), d(s^0, v^1)]$.

Exercise 3 - Solution

Correctness:

Main Claim:

1. There exists a path of cost c from s to v in G iff there exists a path of cost c from s^0 to v^0 in G'

2. There exists an *almost valid* path of cost c from s to v in G iff there exists a path of cost c from s^0 to v^1 in G'

Given the claim, the correctness of the algorithm follows from the fact that we return $\min[d(s^0, v^0), d(s^0, v^1)]$ **for each node.**

Exercise 3 - Solution

Proof of claim:

1. Since the first copy is identical to G, all paths V^0 are identical to paths in G.

2. Let p be an almost-valid path from s to v in G. Let (a,b) be the edge used in opposite direction, then, by the construction, G' includes the path s^0 ----- b^0-a^1----- v^1.

From the construction of G' we know that there is a path in G, s ----- b with the same cost as the subpath from s^0 ----- b^0. In the same way, there is a subpath in G from a to v with the same cost as the subpath from a^1 ---- v^1. Since the cost of the edge b^0 - a^1 in G' is the same as the cost of a – b in G, the total cost of s^0 ----- b^0 - a^1 ---- v^1 is the same as the cost of the equivalent almost-valid path s ----- b – a ----- v in G.

Direction 2: Let s^0 ----- b^0-a^1----- v^1 be a path in G'. Using similar arguments as in the first direction, s ----- b -a ----- v is a path in G, so s ----- b-a ----- v is almost-valid.

Exercise 3 - Solution

Time complexity:

1. Constructing G': $n' = 2n, \quad m' = 3m$ and therefore $O(n' + m') = O(n + m)$.
2. Running Dijkstra on G': $O(m' + n' \cdot \log n') = O(m + n \cdot \log n)$.
3. Returning the result for each $v \in V$: $O(n') = O(n)$.

Exercise 4

Let $G = (V, E)$ be a weighted directed graph with positive weights except one edge s.t $w(a, b) < 0$. Suggest an algorithm that finds the shortest paths from $s \in V$ to all vertices in G.

Idea: Calculate separately the length of a shortest path that does not use (a, b) and that must use (a, b).

Algorithm:

1. Construct a new graph G' such that $G' = (V, E \backslash \{(a, b)\})$
2. Run Dijkstra on G' from s. Save output as d_1
3. Run Dijkstra on G' from b. Save output as d_2
4. For each $v \in V$, output $\min[d_1(v), d_1(a) + w(a, b) + d_2(v)]$

Exercise 4 - Solution

Correctness Claim:

If a path from s to v does not contain (a, b), then by correctness of Dijkstra, d_1 holds the correct shortest path from s to v.

Else, then the path is given by $s \to a \to b \to v$.

The shortest path is composed of sub-shortest paths. We know that:

1. $s \to a$ is a shortest path (Found by Dijkstra on G').
2. $b \to v$ is a shortest path (Found by Dijkstra on G').
3. $a \to b$ is the only negative edge and therefore must be a shortest path from a to b

Time Complexity:

Building the graph takes $O(n + m)$ and running Dijkstra twice takes $O(m + n \cdot \log n)$.